

Cloud-Based Real-Time Chat Application Using Serverless Backend Architecture

Project: NexaChat | Organization: Pinsout, Noida

Abstract

This report describes NexaChat, a secure real-time chat application built with Next.js, TypeScript, Tailwind CSS, Firebase Authentication, Cloud Firestore, Firebase Storage, and Firebase Hosting. The system uses managed serverless services to reduce infrastructure management and support scalable realtime communication.

Introduction

NexaChat is a cloud-based real-time chat application designed to provide secure messaging, group communication, user profiles, dark mode, message actions, and responsive access across desktop and mobile devices.

Objectives

Develop a scalable real-time chat application using serverless architecture, reduce infrastructure management, support authentication, realtime messaging, groups, profiles, settings, and prepare performance and scalability documentation.

Problem Statement

Traditional chat systems require managed servers, socket infrastructure, scaling planning, and manual operational monitoring. The project solves this by using managed serverless services for authentication, realtime sync, storage, hosting, and security rules.

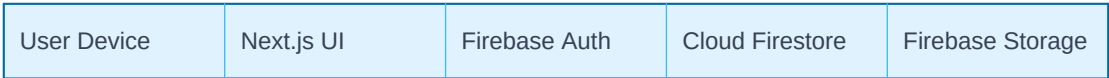
Existing System

Traditional hosting depends on a dedicated backend server, manually managed WebSocket services, server database connections, and capacity planning. This increases operational cost and complexity for student and small-team deployments.

Proposed System

The proposed system uses Next.js with Tailwind CSS on the frontend and Firebase services for authentication, realtime database synchronization, storage, and hosting. Protected routes and Firebase rules enforce application security.

System Architecture Diagram



User Device -> Next.js UI -> Firebase Authentication -> AuthGuard -> Cloud Firestore realtime listeners -> Firebase Storage for media -> Firebase Hosting and monitoring.

Technology Stack

Next.js 15, React, TypeScript, Tailwind CSS, Firebase Authentication, Cloud Firestore, Firebase Storage, Firebase Hosting, Sonner toast notifications, and Artillery or k6 for performance testing.

Features Implemented

Authentication, forgot/change password, realtime messaging, group chats, reactions, replies, message edit/delete, seen list, message status, chat search, profile, settings, dark mode, friend requests, archived chats, muted chats, and blocked users.

Firebase Architecture

Firebase Auth manages identity. Firestore stores users, chats, groups, messages, requests, presence, and user-scoped preferences. Storage handles attachments. Firebase rules protect reads and writes by user identity and membership.

Authentication Flow

User signs in or registers through Firebase Authentication. Protected pages are wrapped in AuthGuard. If no authenticated user is present, the app redirects to /auth/login.

Real-Time Messaging Flow

The chat room listens to the active message collection with Firestore realtime listeners. Sending a message validates and sanitizes input, writes to Firestore, updates the chat metadata, and automatically syncs to participants.

Database Schema

Main collections: users, presence, friends, friendRequests, chats/messages, groups/messages, archivedChats, deletedChats, mutedChats, blockedUsers, notificationState, and forwardQueue.

Screenshots

Add final screenshots for login, register, chat list, chat room, group chat, profile, settings, friends, blocked users, and mobile responsive views after deployment.

Serverless Benefits

No dedicated server management, automatic scaling, managed authentication, managed realtime database, simplified deployment, cost efficiency for variable workloads, and integrated monitoring through Firebase and Google Cloud tools.

Performance Testing

Use Artillery or k6 to test concurrent message creation, chat list loading, group message fan-out, and listener responsiveness. Measure latency, error rate, and successful message delivery under load.

Comparison: Serverless vs Traditional Hosting

Serverless reduces infrastructure work and scales automatically, while traditional hosting offers more low-level control but requires server provisioning, socket scaling, database connection management, and manual monitoring.

Future Scope

Push notifications, end-to-end encryption, media compression, advanced moderation, cloud functions for notifications, analytics dashboard, emulator-based rule tests, and larger-scale performance benchmarking.

Conclusion

NexaChat demonstrates a scalable serverless chat system using modern frontend tooling and Firebase managed services. The architecture supports realtime synchronization, protected user workflows, and efficient deployment.

References

Firebase Documentation, Cloud Firestore Security Rules Documentation, Next.js Documentation, Tailwind CSS Documentation, Google Cloud/Firebase Hosting Documentation, Artillery and k6 Documentation.

Final Readiness Checklist

- ? Protected route layouts added for chats, profile, settings, friends, and groups.
- ? Native browser alerts, confirms, and prompts replaced with Sonner toasts.
- ? Mobile overflow protection and responsive message media sizing added.
- ? Input validation and sanitization added for messages and groups.
- ? Firestore and Storage security rules included.
- ? TypeScript checks pass.
- ? Performance testing, screenshots, and emulator rule tests remain before final submission.